Projet individuel – Architecture de données SmartCampus

TD 2 – Du MCD au MLD / MPD SQL

Durée : 1 h 30

Modélisation relationnelle et technique

1. Contexte et problématique

À la suite du premier atelier, vous disposez d'un MCD représentant les données nécessaires au fonctionnement de SmartCampus.

L'équipe de développement souhaite maintenant préparer l'implémentation de la base de données relationnelle.

Votre mission consiste à transformer votre modèle conceptuel en un modèle relationnel, puis à définir les caractéristiques techniques nécessaires à son implémentation dans un SGBD SQL.

Vous utiliserez PostgreSQL comme moteur de référence.

Le fichier `schema_sql_starter.sql` constitue un point de départ incomplet. Vous devez l'analyser, le compléter et vérifier sa cohérence avec votre modèle.

Problématique :

Comment transformer un modèle métier en une structure SQL techniquement exploitable, tout en garantissant l'intégrité des données et des performances adaptées aux usages ?

2. Travail demandé

Étape 1 – Traduire le MCD en MLD

À partir du MCD produit lors du TD 1, construisez le modèle logique relationnel.

Vous devez appliquer les règles de passage suivantes :

- Une entité devient une table.

- Un attribut devient une colonne.
- Un identifiant devient une clé primaire.
- Une association 1,N est généralement représentée par une clé étrangère du côté N.
- Une association N,N devient une table associative.
- Les associations porteuses d'attributs peuvent nécessiter une table spécifique.
- Les relations historiques doivent être traduites en tables permettant de conserver les différentes occurrences.

Questions pour vous guider :

1. Quelles tables sont nécessaires ?
2. Où placer les clés étrangères ?
3. Quelles associations doivent devenir des tables ?
4. Comment représenter les affectations successives d'un capteur ?
5. Comment garantir qu'un équipement ne possède pas deux localisations simultanément si cette règle est retenue ?

Étape 2 – Définir les types de données

Pour chaque colonne, sélectionnez un type de données adapté à la nature de l'information.

Exemples de types PostgreSQL :

Type	Utilisation	Exemple
BIGINT	Identifiant numérique de grande capacité	id_mesure
INTEGER	Nombre entier	surface en m²
VARCHAR(n)	Texte de longueur limitée	nom_equipement
TEXT	Texte de longueur variable	compte_rendu
DATE	Date sans heure	date_installation

TIMESTAMP	Date et heure	date_mesure
TIMESTAMPTZ	Date et heure avec gestion du fuseau horaire	horodatage d'une mesure
BOOLEAN	Valeur vrai/faux	actif
NUMERIC(p,s)	Valeur décimale précise	consommation
JSONB	Données semi-structurées	configuration

Travail à réaliser :

Pour chaque attribut, indiquez :

- Le type SQL retenu.
- La justification de ce type.
- La possibilité ou non d'accepter une valeur NULL.
- Une éventuelle contrainte de domaine.

Exemple : un ID_Bâtiment doit être stocké sous forme d'un entier numérique et non sous forme de texte.

Étape 3 – Définir les contraintes d'intégrité

Les contraintes permettent de faire respecter les règles métier directement au niveau de la base de données.

Vous devez identifier les contraintes nécessaires parmi les suivantes :

- PRIMARY KEY : Garantit l'unicité et la non-nullité de l'identifiant d'une ligne.
- FOREIGN KEY : Garantit qu'une référence pointe vers une ligne existante dans une autre table.
- NOT NULL: Interdit l'absence de valeur pour un attribut obligatoire.
- UNIQUE : Garantit l'unicité d'une valeur ou d'une combinaison de valeurs.
- CHECK : Permet de contrôler qu'une valeur respecte une condition.

Exemples :

- Un ID_Bâtiment doit obligatoirement être une primary key soit un entier positif non null et unique.

Étape 4 – Identifier les requêtes métier et les index nécessaires

Un index est une structure permettant d'accélérer certaines recherches dans une base de données.

Il fonctionne comme un index de livre : plutôt que de parcourir toutes les lignes, le moteur peut utiliser une structure organisée pour retrouver plus rapidement les informations recherchées.

Cependant, les index consomment de l'espace et augmentent le coût des opérations d'écriture.

Votre mission consiste à identifier les requêtes les plus importantes de SmartCampus et à proposer des index adaptés.

Requête 1 – Consulter les équipements d'un bâtiment

```sql
SELECT e.*

FROM equipement e

JOIN zone z ON e.id_zone = z.id_zone

WHERE z.id_batiment = 10;
```

Question : Quels attributs sont utilisés pour filtrer et réaliser les jointures ?

Requête 2 – Consulter les dernières mesures d'un capteur

```sql
SELECT *

FROM mesure

WHERE id_capteur = 25

ORDER BY date_mesure DESC

LIMIT 10;
```

Question : Comment optimiser la recherche par capteur et le tri chronologique ?

Requête 3 – Consulter les interventions d'un équipement

```sql
SELECT *

FROM intervention

WHERE id_equipement = 100

ORDER BY date_creation DESC;
```

Question : Quels index pourraient accélérer cette consultation ?

Requête 4 – Consulter les interventions d'un technicien

```sql
SELECT i.*

FROM intervention i

JOIN participation p

  ON i.id_intervention = p.id_intervention

WHERE p.id_technicien = 5;
```

Question : Comment optimiser la recherche dans une table associative ?

Requête 5 – Calculer les consommations mensuelles d'un bâtiment

Vous devez proposer une requête permettant de récupérer les consommations associées aux équipements d'un bâtiment sur un mois.

Question : Quelles tables doivent être jointes et quelles colonnes doivent être filtrées ?

Étape 5 – Contrôler les anomalies

Une fois votre schéma SQL défini, vous devez vérifier son comportement.

Testez les situations suivantes :

- Création d'un équipement avec un identifiant de zone inexistant.
- Insertion d'un bâtiment sans nom.
- Création de deux capteurs possédant le même identifiant métier.

- Affectation d'un capteur à un équipement avec une date de fin antérieure à la date de début.
- Création d'une intervention sans équipement associé.

Pour chaque cas, précisez :

- Si l'insertion doit être autorisée ou refusée.
- Quelle contrainte intervient.
- Quel message d'erreur ou comportement est attendu.

3. Livrables attendus

Dossier :

- MLD relationnel.
- Script SQL complété.
- Types et contraintes justifiés.
- Cinq requêtes métier fonctionnelles.
- Index proposés et justification.
- Résultats des tests d'intégrité.
- Analyse des plans d'exécution si l'environnement le permet.

Critères de réussite :

Le schéma doit respecter le modèle conceptuel, garantir les règles d'intégrité et proposer des index cohérents avec les requêtes réellement attendues.